

```

import numpy as np
import pandas as pd
import lightgbm as lgb
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import ndcg_score

def loading_and_preprocessing_of_data(loans="kiva_loans.csv",
theme="loan_themes_by_region.csv", mpi="kiva_mpi_region_locations.csv",
country="country_profile_variables.csv") -> pd.DataFrame:
    # In this step, we are loading the required datasets
    #The latin-1 encoding helps us in avoiding any decoding errors while processing the datasets
we are giving to the model
    # The low_memory=False bit helps in loading the entire datasets in one instance instead of
the default block-by-block loading since some of our datasets contain a high number of
datapoints
    df_theme = pd.read_csv(theme, encoding="latin-1", low_memory=False)
    df_mpi = pd.read_csv(mpi, encoding="latin-1", low_memory=False)
    df_country = pd.read_csv(country, encoding="latin-1", low_memory=False)

    df = df_theme.copy() # this step is for us to ensure that the original
loan_themes_by_region.csv file doesn't get impacted when we make any changes and they
can be operated independently

    # In this step, we are making all the available data in the following two columns standard
and also removing any additional characters such as additional space
    if "ISO" in df.columns:
        df["ISO"] = df["ISO"].astype(str).str.upper().str.strip()
    if "country" in df.columns:
        df["country"] = df["country"].astype(str).str.strip()

    # In this step, we are ensuring that as per our model proposal, the code can incorporate
cross features between the datasets loan_themes_by_region.csv and
country_profile_variables.csv and prevent any error if a column value is missing
    if "country" in df.columns and "country" in df_country.columns:
        df = pd.merge(df, df_country, on="country", how="left", suffixes=("", "_country"))
    #performs a left join like function on country as the matching column between the two
datasets

    # Similar step as above based on Location name or regions
    if "LocationName" in df.columns and "LocationName" in df_mpi.columns:
        df = pd.merge(df, df_mpi[["LocationName", "MPI", "lat", "lon"]], on="LocationName",
        how="left", suffixes=("", "_mpi"))

```

```

elif "region" in df.columns and "region" in df_mpi.columns:
    df = pd.merge(df, df_mpi[["region", "MPI", "lat", "lon"]], on="region", how="left",
suffixes=("", "_mpi"))

```

Since our model is Gradient Boosted Decision Tree, it's important to ensure that the numerical features of the datasets are cleaned up. Therefore, text representations are converted into numerical representations and missing values are filled with Median values to avoid any errors later

```

number_of_columns = ["amount", "rural_pct", "MPI", "lat", "lon", "Population density (per
km2, 2017)", "GDP per capita (current US$)"]

```

```

for column in number_of_columns:

```

```

    if column in df.columns:

```

```

        df[column] = pd.to_numeric(df[column], errors="coerce")

```

```

        df[column] = df[column].fillna(df[column].median())

```

This section is to enable Layer 1 of our algorithm to provide numerical inputs for decision trees and also label encoding for text columns in the data frame.

```

category_columns = ["sector", "Loan Theme Type", "country", "Region"]

```

```

for column in category_columns:

```

```

    if column in df.columns:

```

```

        df[column] = df[column].fillna("Unknown").astype(str)

```

```

        le = LabelEncoder()

```

```

        df[f"{column}_enc"] = le.fit_transform(df[column])

```

```

return df

```

#In this section, cross features are being created through relationship between lender preferences and characteristics of the loans which are a core requirement for our Layer 1

```

def develop_cross_features_set(df: pd.DataFrame) -> pd.DataFrame:

```

```

    print("Cross-features for LightGBM Ranker for Layer 1 being developed")

```

For the calculation of match scores $S(u,i)$, lender preferences with loan and region specific attributes are cross featured. Eventually, this block of code is used to train the LightGBM model for ranking

```

    np.random.seed(100)

```

```

    df["lender_serial_number"] = np.random.randint(1, 1001, size=len(df)) #Select from a base of 1000 IDs

```

```

    df["risk_tolerance"] = np.random.choice(["Low", "Medium", "High"], size=len(df))

```

This block creates a new feature leveraging cross featuring between lender risk preference and regional poverty index (MPI)

```

if "MPI" in df.columns:
    df["risk_mpi_engagement"] = df["MPI"] * df["risk_tolerance"].map({"Low": 0.5,
"Medium": 1.0, "High": 1.5})
else:
    df["risk_mpi_engagement"] = 1.0

# This section helps with ranking for the algorithm to define the level of granularity we
would like for the ranking
if "amount" in df.columns:
    amt = df["amount"]
    df["relevance"] = pd.qcut(amt.rank(method="first"), q=10, labels=[0, 1, 2, 3, 4, 5, 6, 7, 8,
9]).astype(int)
else:
    df["relevance"] = np.random.randint(0, 10, size=len(df))

df = df.sort_values(by="lender_serial_number").reset_index(drop=True)
return df

```

#In the following section, the Layer 1 ranking modelling process is starting by defining the features

```

def training_for_layer1_ranking(df: pd.DataFrame):

    print("Training process for the Layer 1 started for LightGBM Ranking algorithm")

    feature_columns = [
        "amount", "rural_pct", "MPI", "lat", "lon",
        "sector_enc", "Loan Theme Type_enc", "country_enc",
        "risk_mpi_engagement"
    ]

    features = [f for f in feature_columns if f in df.columns]

    X = df[features]
    y = df["relevance"]

    # Grouping process through using the attribute lender_serial_number
    query_counts = df.groupby("lender_serial_number").size().to_list()

    train_size = int(0.8 * len(query_counts))
    train_queries = query_counts[:train_size]
    validation_queries = query_counts[train_size:]

```

```

train_row_limit = sum(train_queries) # This is an important variable for our model

X_train, X_val = X.iloc[:train_row_limit], X.iloc[train_row_limit:]
y_train, y_val = y.iloc[:train_row_limit], y.iloc[train_row_limit:]

ranker = lgb.LGBMRanker(
    objective="lambdarank",
    metric="ndcg",
    ndcg_eval_at=[100], #top 100 loans recommended
    learning_rate=0.08, #ensuring that learning is not slowing down in order to prevent any
                        #type of overfitting
    n_estimators=100, #for the model to create up to 100 decision trees
    num_leaves=31, #standard number for LGBMRanker algorithm for controlling the
                  #complexity and depth of individual trees
    random_state=100
)

ranker.fit(
    X_train, y_train,
    group=train_queries,
    eval_set=[(X_val, y_val)],
    eval_group=[validation_queries],
    eval_metric="ndcg",
    callbacks=[lgb.early_stopping(stopping_rounds=10)]
)

return ranker, features, X_val, y_val, validation_queries, train_row_limit

```

#In this section we are implementing the Layer 2 of our system to ensure autonomous algorithm fairness and filter within the 5% guardrail

```

class RegionBasedPolicyFilter:

```

```

    def __init__(self, max_variance: float = 0.05, top_k: int = 100):
        self.max_variance = max_variance
        self.top_k = top_k

    def application_of_filter(self, recommendation_slate: pd.DataFrame, protected_attr_col: str
                             = "rural_tier") -> pd.DataFrame:

        slate = recommendation_slate.head(self.top_k).copy()

```

Selection rates in the protected attribute groups are calculated. If any difference is identified, the lowest rank from Layer 1 output is replaced with the top rank from the underrepresented group

```
group_counts = slate[protected_attr_col].value_counts(normalize=True)

if len(group_counts) > 1:
    variance = group_counts.max() - group_counts.min()

    if variance > self.max_variance:
        underrepresented_group = group_counts.idxmin()

        candidates_outside = recommendation_slate.iloc[self.top_k:]
        eligible_subpool = candidates_outside[candidates_outside[protected_attr_col] ==
underrepresented_group]

        if not eligible_subpool.empty:
            best_replacement = eligible_subpool.sort_values(by="pred_score",
ascending=False).iloc[0]

            dominant_group = group_counts.idxmax()
            swap_target_idx = slate[slate[protected_attr_col] == dominant_group].index[-1]
            slate.loc[swap_target_idx] = best_replacement
            print(f"Swapping of lowest ranking output from Layer 1 with top rank from
underrepresented group {variance:.2f} > {self.max_variance}")

        return slate
```

#The evaluation for the entire two layer model is implemented in the section below. Using the trained LightGBM algorithm used for ranking, predictions are generated on the validation set and reviews lender's individual recommendation and applies policy filter as the guardrail for Layer 2

```
def evaluate_pipeline(ranker, features, X_val, y_val, validation_queries, df_val_full):

    print("\nEvaluation of the model performance")
    preds = ranker.predict(X_val)
    df_val_full["pred_score"] = preds

    ndcg_scores = []
    agent_filter = RegionBasedPolicyFilter(max_variance=0.05, top_k=10)
```

```

if "rural_pct" in df_val_full.columns:
    df_val_full["rural_tier"] = np.where(df_val_full["rural_pct"] > 50, "Rural", "Urban")
else:
    df_val_full["rural_tier"] = np.random.choice(["Rural", "Urban"], size=len(df_val_full))

grouped = df_val_full.groupby("lender_serial_number")
violation_count = 0
total_batches = 0

for lender_serial_number, group in grouped:
    sorted_group = group.sort_values(by="pred_score",
ascending=False).reset_index(drop=True)
    if len(sorted_group) >= 10:
        total_batches += 1
        filtered_slate = agent_filter.application_of_filter(sorted_group,
protected_attr_col="rural_tier")

        # Verification if there is variance after the filter has been applied
        rates = filtered_slate["rural_tier"].value_counts(normalize=True)
        if len(rates) > 1 and (rates.max() - rates.min()) > 0.05:
            violation_count += 1

guardrail_adherence_rate = (1 - (violation_count / max(total_batches, 1))) * 100
print(f"The adherence of the set guardrail is: {guardrail_adherence_rate:.2f}%")

#In the following section, we are running our entire model from inputting the raw data
through all the steps leading towards the final evaluation phase
if __name__ == "__main__":
    df_processed = loading_and_preprocessing_of_data()

    df_featured = develop_cross_features_set(df_processed)

    ranker, features, X_val, y_val, validation_queries, train_row_limit =
training_for_layer1_ranking(df_featured)

    df_val_full = df_featured.iloc[train_row_limit:].copy()
    evaluate_pipeline(ranker, features, X_val, y_val, validation_queries, df_val_full)

```